C3-Solution

A About the Universe

难度	考点
1	基本位运算

题目分析

难度	考点
1	位运算

题目分析

本题只考了位运算的六种符号。

考虑到输入是有符号整数，所以直接输入并输出即可。

我们的平台不会将左移 31 位判错。

示例代码

```c
#include <stdio.h>
int main() {
    int T;
    scanf("%d", &T);
    while (T--) {
        int a, b;
        scanf("%d%d", &a, &b);
        printf("%d %d %d %d %d %d\n", ~a, a & b, a | b, a ^ b, a << b, a >> b);
    }
    return 0;
}
```

B Believe the Rainbow [STD 13]

难度	考点
1	二进制、位运算

题目分析

难度	考点
1	位运算

题目分析

如何取得某个数二进制中的第 i 位是 0 还是 1？

可以先将其右移 i 位，然后和 1 做按位与。

也就是 `(x >> i) & 1`。

示例代码

```c
#include <stdio.h>

int main() {
    int T;
    scanf("%d", &T);
    while (T--) {
        int x;
        scanf("%d", &x);
        for(int i = 31; i >= 0; i--) {
            printf("%d", (x >> i) & 1);
        }
        printf("\n");
    }
    return 0;
}
```

C GPA - 2026 OZDY

难度	考点
2	整数向上取整、浮点数运算

题目分析

要注意的点有如下：

1. 注意 GPA 在 59 和 60 这里是一个跳跃间断点。GPA 小于等于 59 分就是 0 绩点，否则最少也是 1.00 的绩点。
2. 计算综合成绩时，要么写成 $0.2 \times x_1 + 0.1 \times x_2 + \cdots$ 这种形式，要么如果要写 $\dfrac{x_1}{5} + \cdots$ 这种形式的话，要注意必须写成类似于 `x1/5.0` 这样。因为 x_1 是整数，5 也是整数，因此 `x1/5` 是整除，不保留小数点的。我相信你也不希望教务写程序时帮你所有项目都下取整了。
3. 最终综合成绩需要上取整为整数。在 C 语言里比较方便的是朝零取整，即 `(int)x` 这样。什么叫朝零取整呢？就是 0 在哪里就向哪里取整，对于正数就是向下取整，对于负数就是向上取整。朝零取整有成截断取整，效果相当于直接舍弃小数点。那怎么实现向上取整呢？简单，如果 x 有小数部分，那么朝零取整后肯定小于原数（因为成绩肯定都是大于等于 0 的），朝零取整后再加 1 就是向上取整的结果。

示例代码

```c
#include <stdio.h>
int x1,x2,x3,x4,x;
double tmp,GPA;
int main(){
    scanf("%d%d%d%d",&x1,&x2,&x3,&x4);
    tmp=x1/5.0+x2/10.0+x3/10.0+3.0/5*x4;
    //注意，不能写成 x1/5,3/5*x4，因此这样是整数之间的除法，是整除
    x=(int)tmp;//取 tmp 的整数部分
    if (tmp>x) x=x+1;//向上取整
    if (x<60) printf("0.00");
    else{
        GPA=4-3.0*((100-x)*(100-x))/1600.0;
        printf("%.2lf",GPA);
    }
    return 0;
}
```

D 宇宙冷漠

难度	考点
4	循环、模拟、多组数据

题目分析

对每个回合按照出牌顺序直接模拟。用 `last` 记录本回合最近一张非宇宙冷漠牌的伤害，用 `sum` 记录本回合的总伤害。

读入一张基础伤害为 x 的牌时：

- 如果 $x > 0$，这张牌造成 x 点伤害，并且它成为新的“上一张非宇宙冷漠牌”，所以令 `last = x`。
- 如果 $x = 0$，这张牌的实际伤害等于 `last`，且不能更新 `last`。

一个回合开始时，把 `last` 设为 0。这样，如果开头连续出现若干张宇宙冷漠，它们都会正确造成 0 点伤害。每个回合之间相互独立，所以每进入新回合时都必须重新将 `last` 和 `sum` 置为 0。

单张牌的伤害最大为 10^9，牌的总数最多为 2×10^5，总伤害可能达到 2×10^{14}，超过 `int` 范围。因此 `last`、`x` 和 `sum` 都使用 `long long`，读写格式为 `%lld`。

代码不需要保存所有牌，每读入一张就可以立即处理。设所有回合共有 S 张牌，总时间复杂度为 $O(S)$，额外空间复杂度为 $O(1)$。

示例代码

```c
#include <stdio.h>
int main()
{
    int n;
    scanf("%d", &n);
    while (n--)
    {
```

```
        int a;
        scanf("%d", &a);

        long long last = 0;
        long long sum = 0;
        for (int i = 0; i < a; i++)
        {
            long long x;
            scanf("%lld", &x);
            if (x > 0)
            {
                last = x;
            }
            sum += last;
        }
        printf("%lld\n", sum);
    }
    return 0;
}
```

E 合并后屏蔽

难度	考点
2	位运算

题目分析

每个二进制位独立表示一个位置。只要 a 或 b 的对应位中至少有一个 1，合并后该位就为 1，所以合并状态为 `a | b`。两者对应位都为 1 时，合并后仍为 1，不会抵消，也不会产生进位。

在屏蔽状态 c 中，位值为 1 的位置必须关闭。对 c 按位取反后，这些位置变成 0，其余位置变成 1，再与合并状态按位与，即可得到 `y = (a | b) & ~c`。逐位看，结果为 1 当且仅当原来的 c 对应位为 0，并且 a, b 对应位中至少有一个 1，恰好符合题目定义。

依次检查 y 的第 0 至第 31 位。表达式 `(y >> i) & 1U` 将第 i 位移到最低位，并清除其他位，结果为 0 或 1。把这 32 个结果相加，就得到 1 的个数。循环必须包含第 31 位，且每组数据开始时都要把计数器重新置为 0。逐位读取不会改变 y，可以直接输出原结果。

状态值的最大值为 $2^{32}-1=4294967295$，超过 32 位有符号 `int` 的上限，但可以直接使用 `long long` 保存 a, b, c, y，并用 `%lld` 读入和输出。此时仍然使用 `y = (a | b) & ~c`：`a | b` 的第 32 位及以上均为 0，按位与后这些位仍为 0，所以最终结果非负且不超过 $2^{32}-1$，无需额外截断。随后用 `(y >> i) & 1LL` 检查低 32 位即可。

本题并不要求必须使用无符号类型。下面的示例代码选择评测环境中的 32 位 `unsigned int`，对应输入输出格式为 `%u`；改用 `long long` 和 `%lld` 同样正确。计数最大为 32，使用 `int` 和 `%d` 即可。无论选择哪种类型，都要扫描第 0 至第 31 位，并使输入输出格式与变量类型一致。

每组数据只扫描 32 位，时间复杂度为 $O(32)$；共 T 组数据，总时间复杂度为 $O(32T)$。各组可以读入后立即处理并输出，无需数组，额外空间复杂度为 $O(1)$。

示例代码

```
#include <stdio.h>
int main() {
    int T;
    scanf("%d", &T);
    while (T--) {
        unsigned int a, b, c;
        scanf("%u%u%u", &a, &b, &c);
        unsigned int y = (a | b) & ~c;
        int cnt = 0;
        for (int i = 0; i < 32; i++) {
            cnt += (y >> i) & 1U;
        }
        printf("%u %d\n", y, cnt);
    }
    return 0;
}
```

F 加密咕

难度	考点
3	字符输入输出、数组、ASCII 码

题目分析

用两个长度为 26 的字符数组分别保存加密表和解密表。设 `encrypt[i]` 表示字母 `'a' + i` 加密后的小写字母。读入加密表时，用 `%c` 依次读入 26 个字母：

```
scanf(" %c", &encrypt[i]);
```

`%c` 前的空格会跳过前面的换行符等空白字符，不会把上一行结尾读入加密表。

读入一个映射 `encrypt[i]` 后，就可以同时建立反向的解密表：

```
decrypt[encrypt[i] - 'a'] = 'a' + i;
```

这表示如果某个小写字母加密后变成 `encrypt[i]`，那么解密 `encrypt[i]` 时就应该得到原字母 `'a' + i`。由于加密表中的 26 个字母互不重复，这张反向表一定能被完整填好。

读完 `op` 后，输入位置还在第二行的末尾，所以先用 `%c` 读到该行的换行符。之后再用 `%c` 逐个读入第三行的字符。这里的 `%c` 前不能加空格，否则正文开头、结尾和中间的空格都可能被跳过。

对正文中的每个字符 `c`：

- 若它是小写字母，以 `c - 'a'` 作为下标，从对应的表中取出结果。
- 若它是大写字母，以 `c - 'A'` 作为下标，先查出对应的小写结果，再减去 `'a'`、加上 `'A'` 恢复大写。
- 否则不做任何改变，原样输出。

示例代码

```
#include <stdio.h>
int main()
{
    char encrypt[26], decrypt[26];
    for (int i = 0; i < 26; i++)
    {
        scanf(" %c", &encrypt[i]);
        decrypt[encrypt[i] - 'a'] = 'a' + i;
    }
    int op;
    scanf("%d", &op);
    char c;
    while (scanf("%c", &c) != EOF && c != '\n')
    {
        // 读掉 op 所在行剩余的字符
    }
    while (scanf("%c", &c) != EOF && c != '\n' && c != '\r')
    {
        if ('a' <= c && c <= 'z')
        {
            if (op == 0)
                c = encrypt[c - 'a'];
            else
                c = decrypt[c - 'a'];
        }
        else if ('A' <= c && c <= 'Z')
        {
            if (op == 0)
                c = encrypt[c - 'A'] - 'a' + 'A';
            else
                c = decrypt[c - 'A'] - 'a' + 'A';
        }
        printf("%c", c);
    }
    printf("\n");
    return 0;
}
```

G 位运算方程

难度	考点
4	位运算、分类讨论、无符号整数

题目分析

由于这些数的每一位互相独立，因此我们可以单独考虑二进制中的某一位。考虑固定的一位，在该位上

- 数字 `a` 的值为 `A`
- 数字 `b` 的值为 `B`
- 数字 `x` 的值为 `X`

其中，A、B、$X \in \{0, 1\}$。

1. 异或

由于 $A \oplus A = 0$ 且 $0 \oplus X = X$，我们可以对两边同时异或上 A：

$$\begin{aligned} A \oplus X &= B \\ A \oplus A \oplus X &= A \oplus B \\ X &= A \oplus B \end{aligned}$$

所以方程有且仅有唯一的解：$x = a \oplus b$

2. 按位或

分情况：

- 无解情况：`A =1 , B = 0`，`X` 取任何值方程都不成立。这种情况等价于 `A & ~B == 1`。
- 有解情况：`B = 1`，只需 `A | X == 1` 。为了让 `x` 最小，如果 `A` 已经是 `1`，则 `X` 取 `0`；否则 `X` 取 `1`。这种情况只需 `X = B & ~A`，也就是只保留 `b` 中为 `1` 且 `a` 中为 `0` 的位。

因此：

- 无解条件：`(a & ~b) != 0`，或写作 `(a | b) != b`
- 有解时：`x = b & ~a`

3. 按位与

分情况：

- 无解情况：`A = 0 , B = 1`，`X` 取任何值方程都不成立。这种情况等价于 `B & ~A == 1`。
- 有解情况：如果 `B = 0`，为了让 `x` 最小，那么 `X` 取 `0`；如果 `A = 1 , B = 1`，那么 `X` 只能取 `1` 。这种情况只需 `X = B` 。

因此：

- 无解条件：`(b & ~a) != 0`，或写作`(a & b) != b`
- 有解时：`x = b`

示例代码

```c
#include <stdio.h>
int main() {
    int n;
    scanf("%d", &n);
    while (n--) {
        int op;
        unsigned int a, b;
        scanf("%d%u%u", &op, &a, &b);
        if (op == 1) {
            printf("%u\n", a ^ b);
        } else if (op == 2) {
            if ((a | b) != b) {
                printf("-1\n");
            } else {
                printf("%u\n", b & ~a);
            }
```

```
        } else if (op == 3) {
            if ((a & b) != b) {
                printf("-1\n");
            } else {
                printf("%u\n", b);
            }
        }
    }
    return 0;
}
```

H 下一个同亮度编号

难度	考点
5	二进制、位运算、构造

题目分析

两种解法都利用同一个构造原则：找到最低的可行位置，将该位从 0 改为 1，更高位保持不变，再将低位剩余的 1 尽量放到最低端。

解法一：逐位扫描并构造

比较两个不同的二进制整数时，由最高的不同位决定大小。若 $y > x$，在这个最高不同位 i 上，原数必须为 0，新数必须为 1；比它更高的位保持不变。

设原数在第 i 位以下共有 c 个 1。把第 i 位从 0 改成 1 后，为保持总亮灯数不变，低 i 位只能再放 $c-1$ 个 1。因此这一位置可作为变化位，当且仅当它原本是 0，且低位已经出现至少一个 1。

应从低位到高位寻找第一个可行位置。若选择更高的位置作为最高不同位，所得数一定更大：在那个更高的位置，较低变化位的方案仍保留原来的 0，而较高变化位的方案将它置为 1。因此第一个可行位置给出了最优的最高不同位。

位置确定后，为使结果最小，应把剩余的 $c-1$ 个 1 放在最低的 $c-1$ 位，其他低位清零。这部分的数值为 $2^{c-1}-1$。保留第 i 位以上的原值、设置第 i 位、填入最小低位，即可构造答案。

代码扫描时用 `ones` 统计已经经过的低位中 1 的数量。若在既定的 w 位内找不到可行位置，则无解，包括 $x=0$、全为 1，以及所有 1 已集中在最高若干位的情况。不能扩展位宽，也不能把 x 本身当成答案。

状态使用 `unsigned long long`，读写使用 `%llu`，位权常量写为 `1ULL`。最大保留高位的移位量为 60，构造出的数始终小于 $2^w \le 2^{60}$；`ones - 1` 仅在 `ones > 0` 时使用，所有移位量都小于 64。每组时间复杂度为 $O(w)$，额外空间复杂度为 $O(1)$；共 T 组时，总时间复杂度为 $O(Tw_{\max})$，其中 $w_{\max} \le 60$。

解法二：用位运算直接完成构造

当 $x > 0$ 时，设其二进制末尾有 t 个 0，再向左是一段连续的 r 个 1，其中 $r \ge 1$，这一段左侧的下一位是 0。必要时可以暂时在原位宽左侧补一个 0，构造完成后再判断是否超出位宽。原数的局部形态为

$$\text{高位前缀}\ 0\ \underbrace{11\cdots1}_{r\text{ 个}}\ \underbrace{00\cdots0}_{t\text{ 个}}。$$

按补码性质，`low = x & -x` 只保留 x 最低的一个 1，所以 $low = 2^t$。这是因为对 x 按位取反再加一后，末尾的 t 个 0 仍为 0，第 t 位仍为 1，比它更高的位与原数相反；两者按位与后，仅剩第 t 位。

计算 `next = x + low`，加法会从最低的 1 开始连续进位，将这 r 个 1 清零，并把它们左侧的 0 置为 1，更高位保持不变。这恰好完成了解法一中寻找最低可行变化位并进位的步骤。

接下来还需要把 $r-1$ 个 1 放到最低端。`x & ~next` 恰好取出进位时被清掉的那段 1，其数值为 $(2^r-1)2^t$；除以 `low` 后得到 2^r-1，再右移一位，得到 $2^{r-1}-1$。这就是最低 $r-1$ 位全为 1 的数，与 `next` 按位或即可得到答案：

```
long long low = x & -x;
long long next = x + low;
long long y = (((x & ~next) / low) >> 1) | next;
```

例如，取 $w=6$、$x=28$，构造过程为：

```
x          = 011100
low        = 000100
next       = 100000
x & ~next  = 011100
补回的低位 = 000011
y          = 100011
```

所得 $y=35$，仍有三个 1，且是严格大于 28 的最小合法编号。由于该方法与解法一选择相同的变化位，并将相同数量的 1 放在最低端，其正确性和最小性由前述构造证明保证。

当 $x=0$ 时没有最低的 1，也不存在亮位数相同的更大整数，应直接输出 `-1`，同时避免除以零。对于 $x>0$，若所得 $y \geq 2^w$，说明最低可行的变化位已经超出允许位宽，因此无解；否则输出 y。这也覆盖了全为 1 或所有 1 已集中在最高若干位的情况。

本题有 $0 \leq x < 2^{60}$，可以使用有符号 `long long`。`-x` 可表示，`low` 为正数，`next` 不超过 2^{60}，补回低位后的结果小于 2^{61}，都不会发生有符号溢出；参与右移的商非负。判断上界时使用 `1LL << w`，其中 $w \leq 60$，不能写成 `1 << w`。在固定的 64 位整数模型下，每组只需常数次运算，时间复杂度为 $O(1)$，额外空间复杂度为 $O(1)$；共 T 组时总时间复杂度为 $O(T)$。

示例代码

解法一：逐位扫描并构造

```
#include <stdio.h>
int main() {
    int T;
    scanf("%d", &T);
    while (T--) {
        int w;
        unsigned long long x;
        scanf("%d%llu", &w, &x);
        int ones = 0, found = 0;
        unsigned long long y = 0;
        for (int i = 0; i < w; i++) {
            if ((x >> i) & 1ULL) {
                ones++;
            } else if (ones > 0) {
                unsigned long long high = (x >> (i + 1)) << (i + 1);
```

```
                y = high | (1ULL << i) | ((1ULL << (ones - 1)) - 1ULL);
                found = 1;
                break;
            }
        }
        if (found) {
            printf("%llu\n", y);
        } else {
            puts("-1");
        }
    }
    return 0;
}
```

解法二：位运算直接构造

```
#include <stdio.h>
int main() {
    int T;
    scanf("%d", &T);
    while (T--) {
        int w;
        long long x;
        scanf("%d%lld", &w, &x);
        if (x == 0) {
            puts("-1");
        } else {
            long long low = x & -x;
            long long next = x + low;
            long long y = (((x & ~next) / low) >> 1) | next;
            printf("%lld\n", y < (1LL << w) ? y : -1LL);
        }
    }
    return 0;
}
```

I Tensorfactory

难度	考点
6	数组，模拟，坐标转换

题目分析

张量的形状会改变，但底层数据的一维顺序始终不变。因此，只需维护：

- 一个长度为 N 的一维数组 `data`；
- 当前维度数 c；
- 当前各维长度 $D_1, D_2, \ldots, D_c$。

对于一个合法坐标 $(x_1, x_2, \ldots, x_c)$，可以用循环计算其一维下标：

```
index = 0
依次处理 i = 1, 2, ..., c:
    index = index * D_i + x_i
```

最终得到

$$index = (((x_1 D_2 + x_2) D_3 + x_3) \cdots) D_c + x_c。$$

由于只有在坐标合法时才继续计算，始终有 $0 \le index < N \le 10^6$，不会发生溢出。

查询与修改

先判断输入的维度数是否等于当前维度数。若不相等，直接判为非法；否则依次检查每个坐标是否满足 $0 \le x_i < D_i$。

若操作合法，则计算一维下标。查询操作输出 `data[index]`，修改操作给 `data[index]` 赋新值且不输出。非法操作统一输出 `-1`。

即使维度不匹配，也必须先读完该操作行中给出的全部 c 个坐标；否则会影响后续输入的读取。

重塑

重塑合法当且仅当新形状的各维长度之积等于固定的元素总数 N。合法时只替换当前维度数和形状，不修改 `data`。

直接连乘可能溢出。由于每个新维度长度都为正数，在执行 `product *= D'_i` 前检查：

```
product > N / D'_i
```

若条件成立，则乘积必然大于 N，当前操作一定非法，不要再连乘导致溢出反而变“合法”了。

判断重塑操作是否合法时可能发生乘法溢出。在每次乘法前，算法检查当前乘积是否大于 N / D'_i。若成立，新乘积必然大于 N，不可能等于 N，因此判为非法。否则新乘积不超过 N，乘法安全。全部处理后，乘积等于 N 当且仅当重塑合法。证毕。

示例代码

```c
#include <stdio.h>
int k, n, q, d[16], p[16], a[1000010];
int main()
{
    scanf("%d", &k);
    n = 1;
    for (int i = 0; i < k; i++)
    {
        scanf("%d", &d[i]);
        n *= d[i];
    }
    for (int i = 0; i < n; i++)
        scanf("%d", &a[i]);
    scanf("%d", &q);
    while (q--)
    {
        int op, c;
        scanf("%d%d", &op, &c);
```

```
        if (op == 1)
        {
            int pos = 0, ok = (k == c);
            int x;
            for (int i = 0; i < c; i++)
            {
                scanf("%d", &x);
                pos = pos * d[i] + x;
                if (x < 0 || x >= d[i])
                    ok = 0;
            }
            printf("%d\n", ok ? a[pos] : -1);
        }
        if (op == 2)
        {
            int pos = 0, val = 0, ok = k == c;
            for (int i = 0, x; i < c; i++)
            {
                scanf("%d", &x);
                pos = pos * d[i] + x;
                if (x < 0 || x >= d[i])
                    ok = 0;
            }
            scanf("%d", &val);
            if (ok)
                a[pos] = val;
            else
                printf("-1\n");
        }
        if(op == 3)
        {
            int prd = 1;
            for (int i = 0; i < c; i++)
            {
                scanf("%d", &p[i]);
                // 仔细观察下一行的逻辑
                prd = (n / p[i] < prd ? n + 1 : prd * p[i]);
            }
            if (prd != n)
                printf("-1\n");
            else
            {
                k = c;
                for (int i = 0; i < c; i++)
                    d[i] = p[i];
            }
        }
    }
    return 0;
}
```

J kewulalala做实验

难度	考点
7	按位考虑，异或性质

题目分析

异或运算每一位是独立的。操作量 x 可以写成

$$x = \sum_{i \geq 0} 2^i \, [x \text{ 的第 } i \text{ 位是 } 1].$$

也就是说，第 i 位每做一次操作，就会给总花费贡献 2^i。设第 i 位最少需要操作 w_i 次，总答案就是

$$res = \sum_{i=0}^{30} 2^i w_i.$$

因此只需要对每一位单独求出 w_i，再乘 2^i 累加。

固定看某一位，所有数都变成 $0/1$。一次操作只有三种本质效果：

- `00 -> 11`：产生两个相邻的 `1`；
- `11 -> 00`：消掉两个相邻的 `1`；
- `01 -> 10`、`10 -> 01`：交换相邻的 `0` 和 `1`，也就是让一个 `1` 向旁边移动一格。

因此对某一位，就是在一个环上交换不同元素和取反相同元素。最终有两种选择：全部变成 `0`，或全部变成 `1`。

先看全部变成 `0`：要把所有的 `1` 都消掉，必须两两配对。设这些 `1` 的位置按环上顺序为 $p_1 < p_2 < \cdots < p_k$。每消掉一对 `1`，就是把它们通过交换移到相邻位置，再执行一次 `11 -> 00`。若两个 `1` 在环上的距离是 d，那么把它们配对消掉的代价就是 d。对于位置 $u < v$ 的两个 `1`，不绕回时距离是 $v - u$，绕回时距离是 $u + n - v$。因此我们希望让这些 `1` 两两配对，使总距离最小。

设有 k 个 `1`：

若 $k = 0$，已经全是 `0`，代价为 0。

若 k 是奇数，则无论如何都会剩下一个 `1`，不能全部变成 `0`；这时只能尝试全部变成 `1`。

当 k 是偶数时，最优配对一定是按环上顺序相邻配对。因为如果两个配对的区间交叉或嵌套，把它们改成相邻配对不会更差。于是只有两种可能：

1. 顺序配对 $(p_1, p_2), (p_3, p_4), \ldots, (p_{k-1}, p_k)$，代价为

$$c_1 = \sum_{j=1}^{k/2} (p_{2j} - p_{2j-1}).$$

2. 错开配对 $(p_2, p_3), (p_4, p_5), \ldots, (p_k, p_1)$，代价为

$$c_2 = (\sum_{j=2}^{k/2} (p_{2j-1} - p_{2j-2})) + (p_1 + n - p_k).$$

第二种是错开一位配对，其中最后一对 (p_k, p_1) 要从 p_k 绕一圈回到 p_1，距离是 $p_1 + n - p_k$。

全部变成 `1` 的做法完全相同，只要把原来的 `0` 看成要消掉的点，记录所有 `0` 的位置再配对即可。

若当前选择消掉值 t（$t=0$ 或 1），把对应位置记为 $p_1<p_2<\cdots<p_{k_t}$，并记两种代价为 $c_1(t),c_2(t)$，则

$$w_i(t)=\begin{cases}\min(c_1(t),c_2(t)), & k_t \text{ 为偶数},\\ +\infty, & k_t \text{ 为奇数}.\end{cases}$$

因此第 i 位的最少操作次数为

$$w_i=\min(w_i(0),w_i(1)).$$

第 i 位对总答案的贡献是 $2^i w_i$，最终答案为

$$res=\sum_{i=0}^{30}2^i w_i.$$

代码中 $t=0,1$ 分别表示“把当前位为 t 的位置消掉”：消掉 `0` 就是最终全变成 `1`，消掉 `1` 就是最终全变成 `0`。两种选择都可行时取较小值；如果某一位两种都不可行，就输出 `-1`。

时间复杂度为 $O(31n)$，空间复杂度为 $O(n)$。

示例代码

```
#include <stdio.h>
int n, a[100010], b[100010];
int pos[2][100010];
long long res;
int main() {
    scanf("%d", &n);
    for (int i = 1; i <= n; i++) scanf("%d", &a[i]);
    int flag = 0;
    for (int i = 0; i <= 30; i++) {
        int cnt[2] = {0, 0}, ans = 1000000000, tot;
        for (int j = 1; j <= n; j++) {
            b[j] = ((a[j] >> i) & 1);
            pos[b[j]][++cnt[b[j]]] = j;
        }

        for (int t = 0; t < 2; t++) {
            int k = cnt[t];
            if (k & 1) continue;
            if (k == 0) tot = 0;
            else {
                int c1 = 0, c2 = 0;

                //(1,2),(3,4)...配对
                for (int j = 1; j <= k / 2; j++) c1 += pos[t][2 * j] - pos[t][2 *
j - 1];

                //(2,3),(4,5)...,(k,1)回绕配对
                for (int j = 2; j <= k / 2; j++) c2 += pos[t][2 * j - 1] - pos[t]
[2 * j - 2];
                c2 += pos[t][1] + n - pos[t][k];

                tot = c1 < c2 ? c1 : c2;
            }
            if (ans > tot) ans = tot;
```

```
        }

        if (ans == 1000000000) {
            flag = 1;
            break;
        }
        res += ((1ll * ans) << (1ll * i));
    }
    if (flag) puts("-1");
    else printf("%lld", res);
}
```